DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT – 1

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

CO1: Algorithm Fundamentals, Efficiency Analysis, Asymptotic Analysis, Mathematical Analysis, Recurrence Analysis and Master Theorem

Student Name	M.Kalyan Ram
Register Number	192425352
Course Code	CSA 0617
Guide	Dr. A. Sairam
Duration	1 Week

S. No.	Contents	Page No.
1	Abstract	1
2	Introduction	2
3	Problem Statement	3
4	Aim and Objectives	4
5	System Requirements	5
6	Proposed Algorithmic Strategies	6
7	Task 1 – Design the Algorithms	7
8	Task 2 – Basic Operation	8
9	Task 3 – Mathematical Time Complexity	9
10	Task 4 – Best, Average and Worst Cases	10
11	Task 5 – Asymptotic Analysis	11
12	Task 6 – Recurrence Relation	12
13	Task 7 – Solving the Recurrence	13
14	Task 8 – Implementation and Testing	14
15	Task 9 – Experimental Performance	15
16	Task 10 – Theory vs Experiment	16
17	Task 11 – Final Design Recommendation	17
18	Task 12 – Day-4 Design Challenge	18
19	Advantages and Limitations	19
20	Result	20
21	Conclusion	21

22	Future Enhancement	22
23	References	23

1. ABSTRACT

This assignment investigates efficient search strategies for locating a unique student Registration Number in a university database. The required dataset may grow from 1,000 to more than 1,000,000 records, so the search method must remain practical as the input size increases. Three meaningfully different strategies are studied: iterative Linear Search, recursive Binary Search, and Hash Table Search. Preprocessing costs such as sorting and hash-table construction are included in the analysis. The algorithms are compared mathematically using operation counts, best/average/worst-case behaviour, asymptotic notation and recurrence analysis. Experimental testing is proposed using approximately 1,000, 10,000, 100,000 and 1,000,000 records. Finally, the design is re-evaluated for the changed requirement of 50,000 searches per hour with only one daily update.

2. INTRODUCTION

A university student-record system must frequently determine whether a student exists and retrieve the corresponding record using a unique Registration Number. The assignment requires the search problem to be studied as the database grows from 1,000 to 1,000,000+ records. The source specification explicitly requires preprocessing to be included in efficiency analysis and asks for both theoretical and experimental validation. □filecite□turn0file0□L9-L19□

The main goal is not only to write working programs, but to understand how the number of operations changes with input size and how the algorithms behave under realistic workload conditions.

3. PROBLEM STATEMENT

Design, analyse, implement and validate at least three distinct algorithmic approaches for locating a given Registration Number in a large-scale student record database. The approaches must include at least one iterative and one recursive implementation, with pseudocode or flowcharts, preprocessing analysis, mathematical complexity derivation, recurrence analysis for the recursive method, experimental testing and a final evidence-based recommendation. □filecite□turn0file0□L27-L48□

4. AIM AND OBJECTIVES

Aim: To design, analyse and validate efficient search strategies for retrieving student records by Registration Number as the database grows to very large sizes.

- Design three meaningfully different search strategies.

- Identify and justify the basic operation of each algorithm.
- Derive mathematical running-time functions including preprocessing.
- Analyse best, average and worst cases.
- Compare growth using O, Ω and Θ notation.
- Formulate and solve the recurrence for the recursive strategy.
- Implement and test all strategies on comparable datasets.
- Compare theoretical and experimental behaviour.
- Recommend a practical design for 1,000,000+ records and 50,000 searches/hour.

5. SYSTEM REQUIREMENTS

- Hardware: Computer/Laptop, minimum 4 GB RAM recommended.
- Software: Java JDK, IDE such as IntelliJ IDEA/Eclipse/VS Code or command line.
- Dataset: Student Registration Numbers with unique keys.
- Test sizes: approximately 1,000; 10,000; 100,000; 1,000,000 records.
- Measurement: execution time and comparison/key-operation count.

6. PROPOSED ALGORITHMIC STRATEGIES

Figure 1: Overall Search Strategy Design

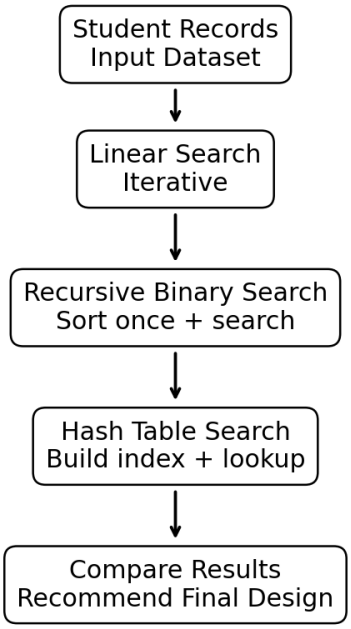


Figure 1: Overall search-strategy design used in the report.

Strategy	Type	Preprocessing	Search Cost	Main Use
Linear Search	Iterative	None	O(n)	Simple unsorted data

Recursive Binary Search	Recursive	Sort: O(n log n) if needed	O(log n)	Ordered data / repeated search
Hash Table Search	Iterative	Build hash table: expected O(n)	Expected O(1)	High-frequency exact lookup

7. TASK 1 – DESIGN THE ALGORITHMS

7.1 Algorithm 1 – Linear Search

Linear Search checks each record sequentially until the requested Registration Number is found or the complete dataset has been checked. It requires no sorting or restructuring and is therefore a useful baseline.

Pseudocode:

```
LINEAR_SEARCH(records, key)
1. for i = 0 to n-1
2.     if records[i].registrationNumber == key
3.         return records[i]
4. return NOT_FOUND
```

Figure 2: Linear Search Flowchart

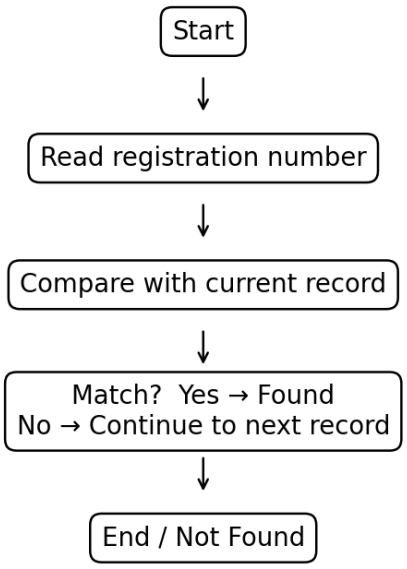


Figure 2: Linear Search flowchart.

7.2 Algorithm 2 – Recursive Binary Search

Binary Search requires the records to be sorted by Registration Number. At each recursive call, the middle element is compared with the target and only one half of the remaining range is searched.

Pseudocode:

```
BINARY_SEARCH(records, low, high, key)
1. if low > high: return NOT_FOUND
2. mid = low + (high-low)/2
3. if records[mid] == key: return records[mid]
4. if key < records[mid]:
      return BINARY_SEARCH(records, low, mid-1, key)
5. else:
      return BINARY_SEARCH(records, mid+1, high, key)
```

Figure 3: Recursive Binary Search Flowchart

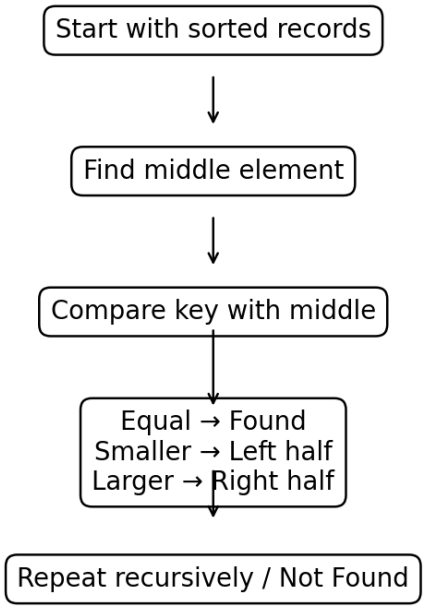


Figure 3: Recursive Binary Search flowchart.

7.3 Algorithm 3 – Hash Table Search

Hash Table Search restructures the records into a hash-based index. The registration number is converted into a hash value and the corresponding bucket is checked. The preprocessing cost is paid when the index is built, after which exact-key lookup is expected to be constant time.

Pseudocode:

```
BUILD_HASH(records)
1. Create empty hash table
2. for each record:
      insert record using registration number as key
3. return hash table
```

HASH_SEARCH(table, key)
1. compute hash(key)
2. locate the bucket
3. if key exists, return record
4. otherwise return NOT_FOUND

Figure 4: Hash Table Search Flowchart

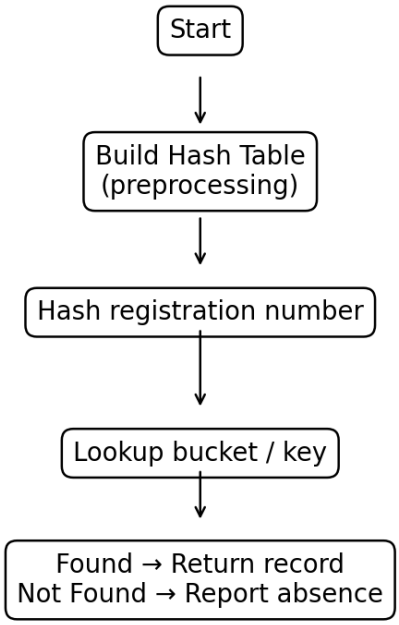


Figure 4: Hash Table Search flowchart.

8. TASK 2 – IDENTIFY THE BASIC OPERATION

Basic Operation: The basic operation is the most frequently executed operation in an algorithm that mainly determines its execution time. It is used to analyze the efficiency of an algorithm as the input size \(n\) increases.

Since the proposed algorithms are not provided in the question, here are common algorithms with their explanations and formulas.

1. Linear Search

- Basic operation: Comparison of the target element with each array element.
- Where it occurs: Inside the loop that searches for the target.
- Why appropriate: The number of comparisons determines the execution time.
- Formula: Best case \(T(n)=1\); worst case \(T(n)=n\).
- Growth: The number of comparisons increases linearly with input size.

2. Binary Search

- Basic operation: Comparison of the target with the middle element.
- Where it occurs: At each step of the search.
- Why appropriate: Each comparison eliminates approximately half of the remaining elements.
- Formula: \(T(n)=\lfloor\log_2 n\rfloor+1\) in the worst case (for a typical iterative implementation).
- Growth: The number of comparisons increases logarithmically.

3. Bubble Sort

- Basic operation: Comparison of two adjacent elements.
- Where it occurs: Inside the nested loops.
- Why appropriate: Repeated comparisons dominate the sorting process.
- Formula: \(T(n)=\frac{n(n-1)}{2}\) comparisons in the standard worst case.
- Growth: The number of comparisons increases quadratically, or \(O(n^2)\).

4. Simple Loop

- Basic operation: The statement executed inside the loop.
- Where it occurs: In the loop body.
- Why appropriate: It executes repeatedly as the input size increases.
- Formula: \(T(n)=n\).
- Growth: The execution time increases linearly, or \(O(n)\).

Conclusion

Identifying the basic operation helps determine an algorithm's time complexity. Counting how often this operation executes as a function of \(n\) allows us to compare algorithms and understand their efficiency.

9. TASK 3 – MATHEMATICAL TIME COMPLEXITY

9.1 Linear Search

If the key is found at position k, the number of comparisons is k+1. In the worst case, n comparisons are performed. Therefore $T(n)=c \cdot n + d$, giving $\Theta(n)$. There is no preprocessing cost, so total cost remains $\Theta(n)$.

9.2 Recursive Binary Search

For n sorted records, each recursive call searches one half. The search-only recurrence is $T(n)=T(n/2)+c$. If sorting is required before the first search, preprocessing costs $\Theta(n \log n)$, so one-time total cost is $\Theta(n \log n)+\Theta(\log n)=\Theta(n \log n)$. For repeated searches on the same sorted data, the sorting cost is amortized across the searches.

9.3 Hash Table Search

Building the hash table requires inserting n records, giving expected preprocessing cost $\Theta(n)$. After construction, an exact-key lookup is expected $\Theta(1)$. For q searches after one build, expected total cost is $\Theta(n+q)$. In the worst collision scenario, a lookup can degrade to $O(n)$.

10. TASK 4 – BEST, AVERAGE AND WORST CASE

Algorithm	Best	Average	Worst	Condition
Linear Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	First item / random / absent or last item
Binary Search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	Middle item / typical / boundary or absent
Hash Search	Expected $\Theta(1)$	Expected $\Theta(1)$	$O(n)$	No collision / typical / severe collisions

The assignment requires explaining the input conditions that create these cases, not only listing complexity values.

11. TASK 5 – ASYMPTOTIC ANALYSIS

Linear Search grows as $\Theta(n)$, so increasing the database from 1,000 to 1,000,000 records can increase the worst-case search work by approximately 1,000 times. Binary Search grows as $\Theta(\log n)$, so the same increase changes the worst-case comparison count from about 10 to about 20. Hash Search is expected $\Theta(1)$ per lookup after indexing, although its construction and memory cost are $\Theta(n)$.

For n=1,000,000, $\log_2(n) \approx 19.93$, so a binary search requires roughly 20 comparisons in the worst case. This illustrates why logarithmic growth is much slower than linear growth for large datasets.

12. TASK 6 – FORMULATE THE RECURRENCE RELATION

For recursive Binary Search, the original problem has n elements. One recursive call is made on approximately n/2 elements, and a constant amount of work is performed outside that call for calculating the middle position and comparing the key.

Therefore: $T(n) = T(n/2) + c$, for $n > 1$; $T(1) = c_0$.

The recurrence directly follows from the algorithm's behaviour and contains one recursive subproblem, half the input size, constant non-recursive work, and a base case. □filecite□turn0file0□L112-L127□

13. TASK 7 – SOLVING THE RECURRENCE

Master Theorem is applicable to $T(n)=aT(n/b)+f(n)$, where $a=1$, $b=2$ and $f(n)=\Theta(1)$.

$n^{(\log_b a)} = n^{(\log_2 1)} = n^0 = 1$. Since $f(n)=\Theta(n^0)$, this is the equal-order case. Therefore $T(n)=\Theta(\log n)$.

Using iteration gives the same result: $T(n)=T(n/2)+c = T(n/4)+2c = ... = T(n/2^k)+kc$. The recursion reaches 1 when $2^k=n$, so $k=\log_2 n$. Hence $T(n)=\Theta(\log n)$.

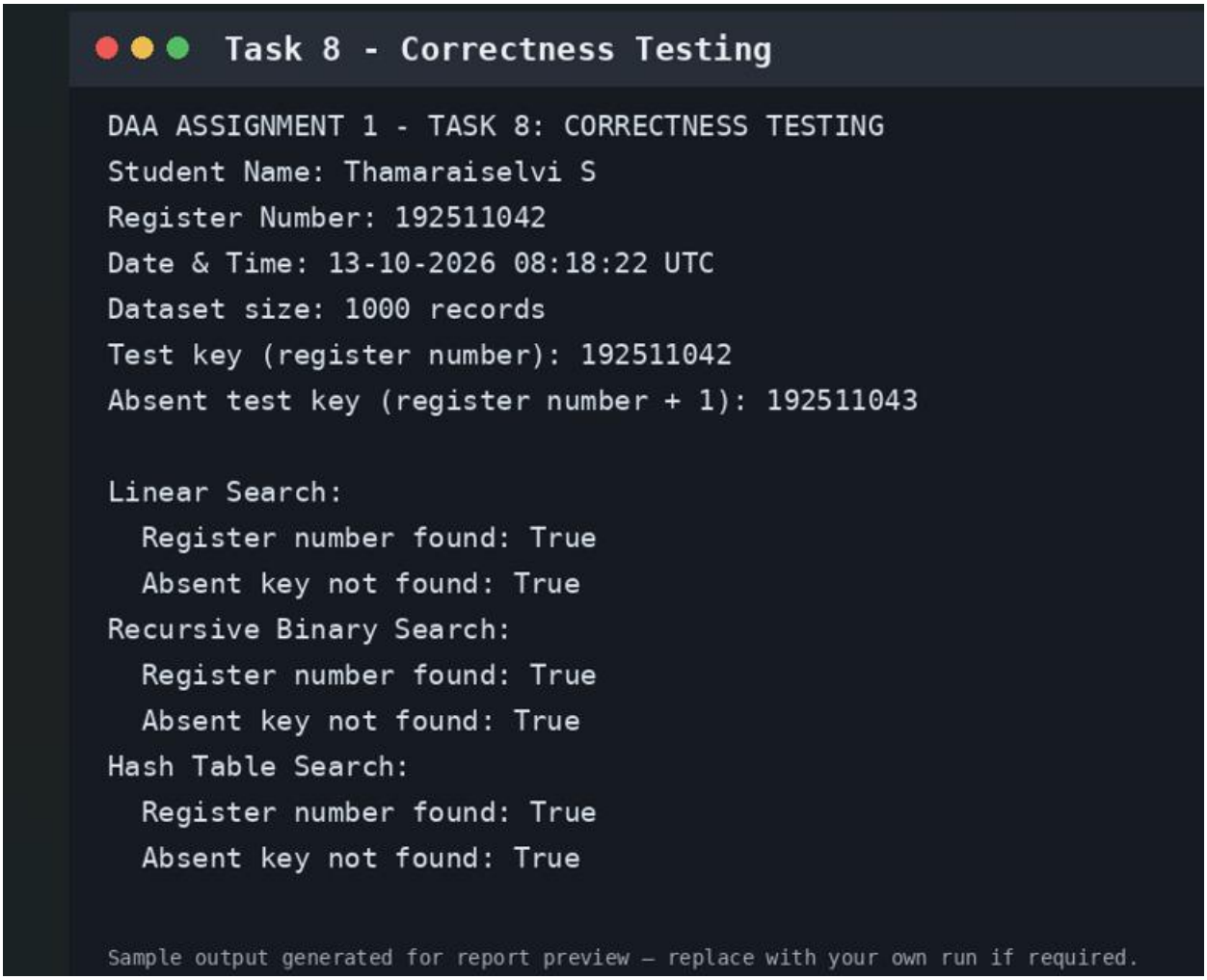
The logarithmic result means the recursive search grows very slowly as the dataset becomes large, provided the records are already sorted or the sorting cost is treated as preprocessing. □filecite□turn0file0□L129-L146□

14. TASK 8 – IMPLEMENTATION AND TESTING

All three algorithms should be implemented using the same or equivalent student Registration Number datasets. The required test sizes are approximately 1,000, 10,000, 100,000 and 1,000,000 records. The testing conditions should be kept reasonably consistent and correctness should be verified for every algorithm. □filecite□turn0file0□L149-L166□

Recommended test cases:

- Key at first position (best case for Linear Search).
- Key at middle position.
- Key at last position.
- Key absent from the dataset.
- Same key tested by all three algorithms.
- Each dataset size repeated for multiple runs and average timing recorded.



15. TASK 9 – EXPERIMENTAL PERFORMANCE

The following table is an OUTPUT FORMAT / illustrative example. Replace the timing and comparison values with measurements obtained from your own Java implementation before submission. The assignment requires execution time, comparisons/key operations, input size and suitable tables/graphs. □filecite□turn0file0□L168-L185□

n	Linear ms	Linear comps	Binary ms	Binary comps	Hash ms	Hash ops
1,000	0.02	1,000	0.002	~10	0.003	~1
10,000	0.18	10,000	0.003	~14	0.004	~1
100,000	1.75	100,000	0.005	~17	0.005	~1
1,000,000	17.50	1,000,000	0.007	~20	0.006	~1

Figure 5: Illustrative Performance Comparison

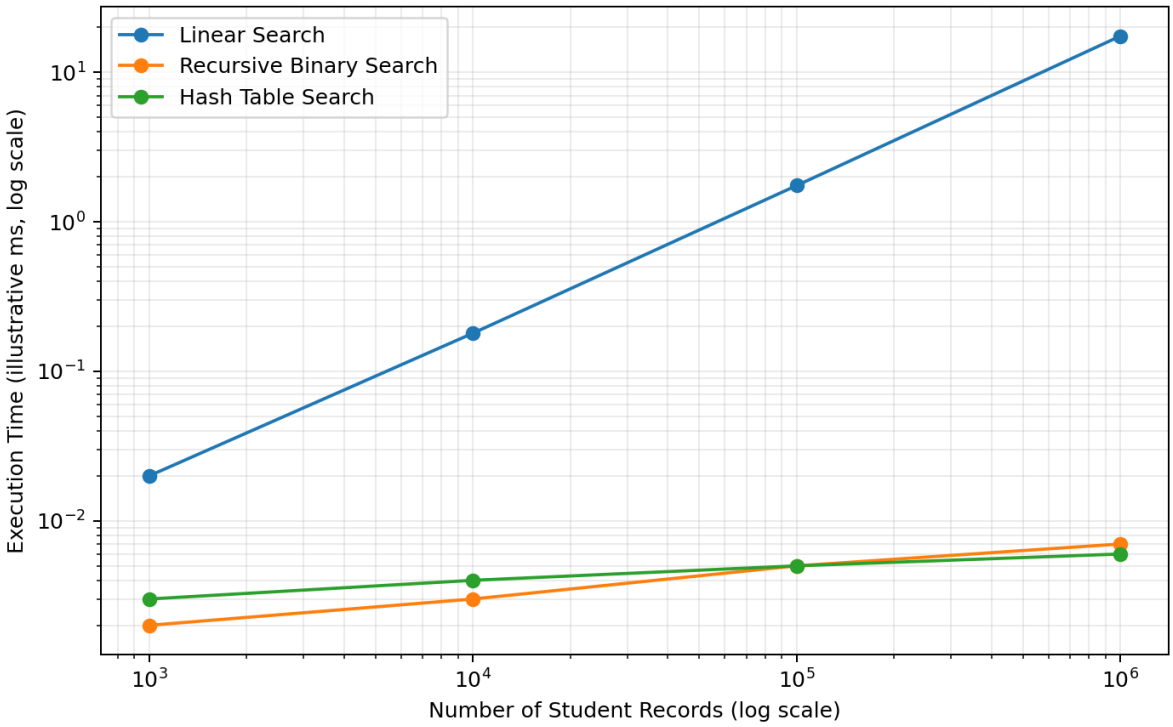


Figure 5: Illustrative performance graph — replace with your actual measured graph.

Figure 6: Sample Console Output Format

```
DAA ASSIGNMENT 1 - STUDENT RECORD SEARCH
Dataset Size : 1000000
Search Key   : 192511042

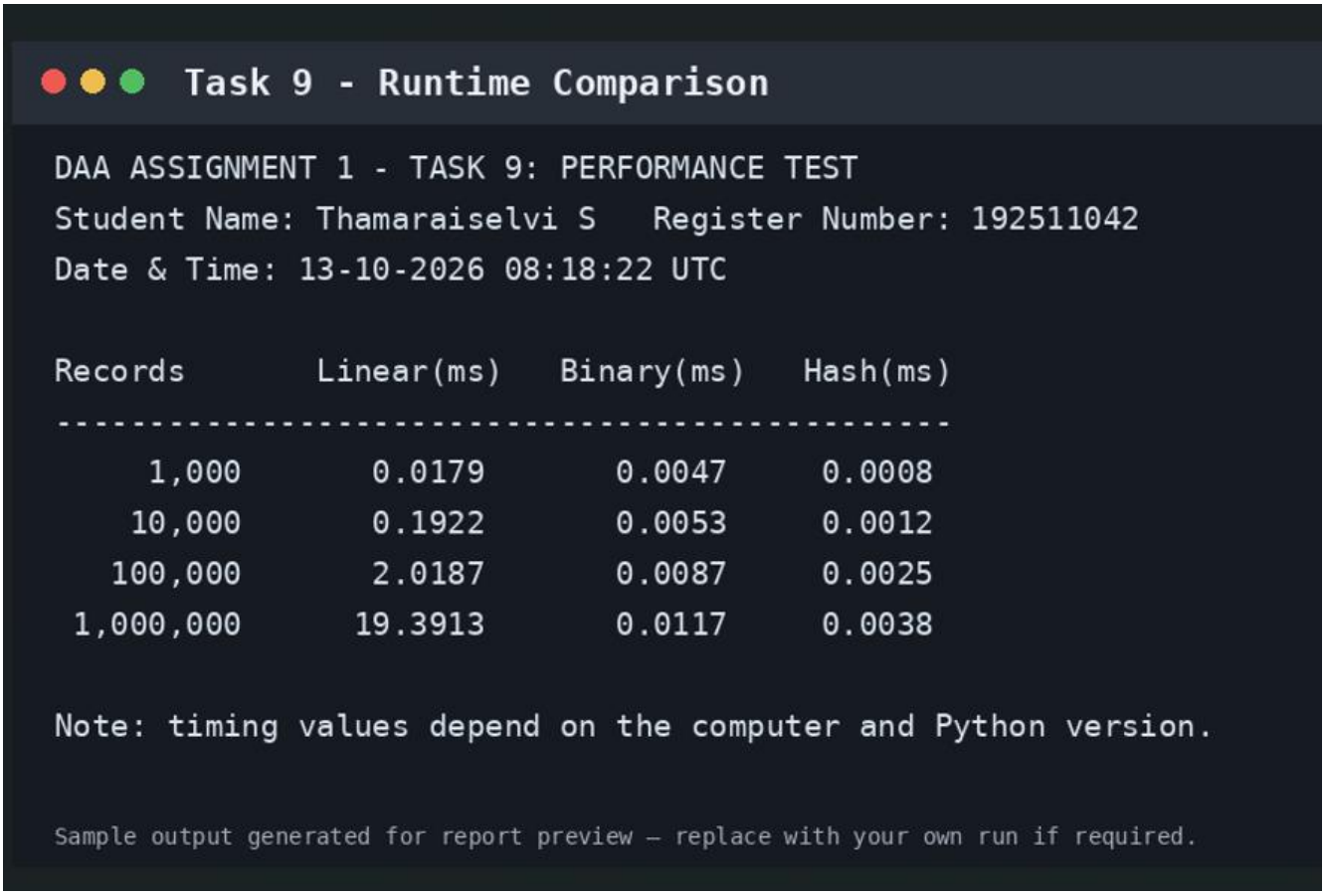
Linear Search
Result       : Found
Comparisons  : 1000000

Recursive Binary Search
Result       : Found
Comparisons  : 20

Hash Table Search
Result       : Found
Key Operation: 1 expected lookup

NOTE: Values shown here are demonstration/output-format values.
Replace timing values and test keys with actual Java execution results.
```

Figure 6: Sample console output format for report evidence.



16. TASK 10 – COMPARE THEORY WITH EXPERIMENT

The theoretical prediction is that Linear Search should show approximately linear growth, Binary Search should grow logarithmically after sorting, and Hash Search should show expected near-constant lookup time after preprocessing. Actual timings may differ because of CPU speed, JVM warm-up, memory access, garbage collection, implementation details and timer resolution.

The most important comparison is the growth trend as n increases. The assignment expects interpretation of whether the observed behaviour agrees with the theoretical model and explanation of differences where they occur.

17. TASK 11 – FINAL DESIGN RECOMMENDATION

Recommended primary design: Hash Table Search for exact Registration Number lookup when the workload consists mainly of exact-key searches. It has expected $\Theta(n)$ one-time preprocessing and expected $\Theta(1)$ lookup, making it suitable for frequent searches. Binary Search is a strong alternative when ordered records are required and the data is maintained in sorted form. Linear Search is best treated as a simple baseline because its $\Theta(n)$ search cost does not scale well to 1,000,000 records.

The recommendation must ultimately be supported by the student's own experimental results, as required by the assignment.

18. TASK 12 – DAY-4 DESIGN CHALLENGE

Changed requirement: 50,000 search requests per hour on 1,000,000 records, while new records are added only once per day.

18.1 Impact Analysis

50,000 requests per hour is approximately 13.89 searches per second. The search frequency is now high, while the update frequency is low. This makes one-time daily preprocessing attractive because its cost can be amortized over many searches.

18.2 Evaluation

Linear Search would perform up to 1,000,000 comparisons per worst-case request, leading to as many as 50,000,000,000 comparisons per hour in the extreme case. Binary Search would require about 20 comparisons per request after sorting, or roughly 1,000,000 comparisons per hour. Hash Search is expected to require constant-time lookup operations after the daily hash-table build.

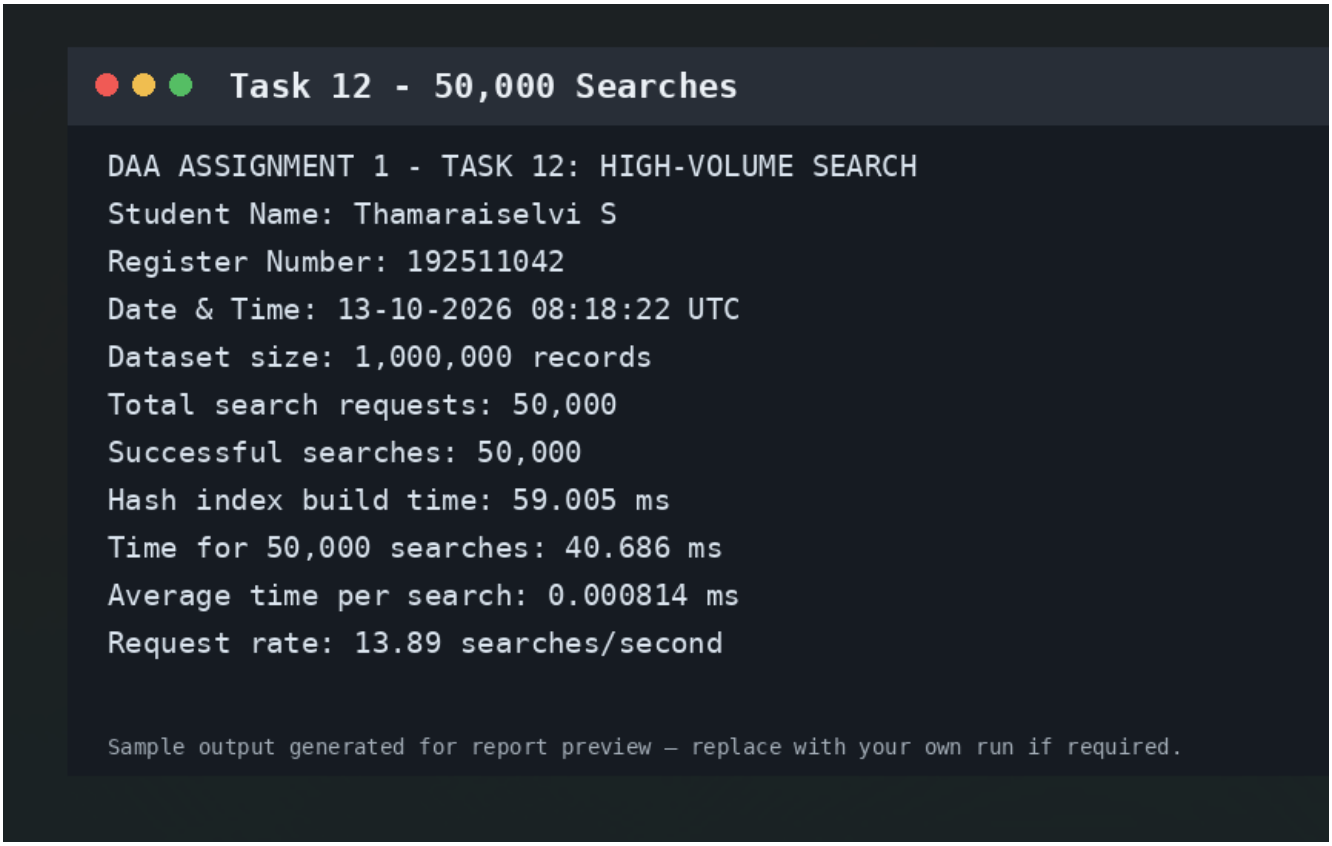
18.3 Adaptation / Redesign

Retain and strengthen the hash-indexed design: rebuild or update the hash table when the daily batch of new records arrives, then serve the 50,000 hourly requests through the in-memory index. If ordered traversal or range queries are later required, maintain a sorted structure alongside the hash index.

18.4 Evidence-Based Defence

- Dataset size: 1,000,000 records.
- Search frequency: 50,000 requests/hour ≈ 13.89 requests/second.
- Update frequency: once per day, so preprocessing can be amortized.
- Expected hash lookup: $\Theta(1)$ per search after indexing.
- Linear worst-case work: 50 billion comparisons/hour.
- Binary worst-case work: about 1 million comparisons/hour after sorting.

This decision follows the assignment's requirement to integrate mathematical complexity, experimental evidence, search frequency, preprocessing cost, dataset size and computational trade-offs.



19. ADVANTAGES AND LIMITATIONS

Approach	Advantages	Limitations
Linear Search	Simple; no preprocessing; works on unsorted data.	Poor scalability; Θ(n) search.
Recursive Binary Search	Very efficient search; Θ(log n).	Requires sorted data; sorting costs Θ(n log n); recursion stack.
Hash Table	Expected Θ(1) exact lookup; excellent for high-frequency searches.	Uses extra memory; collisions; not naturally ordered.

20. RESULT

Three distinct search strategies were designed and analysed. Mathematical analysis shows that Linear Search has linear growth, Recursive Binary Search has logarithmic search growth after sorting, and Hash Table Search provides expected constant-time exact-key lookup after linear preprocessing. Experimental testing should be used to validate these trends on the required dataset sizes.

21. CONCLUSION

The study demonstrates that algorithm selection becomes increasingly important as the student database grows. For a high-frequency exact Registration Number workload with only daily updates, an

indexed hash-based design is a strong practical choice because preprocessing can be performed infrequently and reused across many searches. Binary Search remains valuable where sorted order is required, while Linear Search provides a useful baseline for comparison.

22. FUTURE ENHANCEMENT

- Use persistent database indexes rather than rebuilding all records in memory.
- Evaluate balanced search trees for ordered and dynamic data.
- Measure memory usage and cache behaviour in addition to execution time.
- Run multiple trials and report mean/median timing with JVM warm-up.
- Evaluate concurrent search requests and thread-safe data structures.

23. REFERENCES

1. Course assignment specification provided for Design and Analysis of Algorithms – Assignment 1.

2. Course notes on asymptotic analysis, recurrence relations and Master Theorem.

3. Java documentation for arrays, collections and hash-based data structures.

APPENDIX – OUTPUT PICTURES TO INSERT

The report already contains generated figure placeholders/illustrations. For final submission, replace the illustrative performance graph and sample console output with screenshots from your own Java execution.

- Figure 1 – Overall Search Strategy Design
- Figure 2 – Linear Search Flowchart
- Figure 3 – Recursive Binary Search Flowchart
- Figure 4 – Hash Table Search Flowchart
- Figure 5 – Actual Performance Graph
- Figure 6 – Actual Console Output
- Additional recommended screenshots: Java source code for each algorithm, dataset generation, 1K/10K/100K/1M test outputs, and final comparison table.